

SDS Exercise Sheet 4: Dialogue State Tracking

Nurul Lubis

Release 19.5.2026 — Due 15.6.2026

1 Introduction

In this exercise you are going to explore the DST module of a dialogue system. You will be working with the TripPy [1] codebase.

The objective of this practical are as follows:

- Able to implement parts of a model architecture to perform dialogue state tracking
- Able to train a model to perform state tracking with a reasonable performance

2 TripPy as Dialogue State Tracker

2.1 Gates

TripPy employs a triple-copy strategy to fill the slots. The intuition is that values are either explicitly expressed by the user, that they are expressed by the system and referred to by the user via confirmation or rejection, or that they have been expressed earlier in the dialog as assignment to another domain-slot pair (coreference). Each of these cases is handled by one of three copy mechanisms.

At each turn, to determine which method to use to fill the respective slot, TripPy employ slot gates:

- Slot type gate: What is the value of this slot?
 $\{none, dontcare, true, false, span, inform, refer, request\}$
 - *span*: predict the span and copy the span as the value
 - *inform*: copy the value from the system inform memory
 - *refer*: predict which slot is referred to, and copy the value of that slot
 - *request*: if the slot is a requestable, the value is always “?”
 - otherwise, take the label directly as the slot value
- Span gate: If the slot type is span, where is start and end of that span?
- Refer gate: If the value refers to another slot, which slot is it?

2.2 Loss function

TripPy formulates each gate as classification task and uses cross entropy loss (explained in exercise sheet 3) to optimize it. Because the tracking is divided into 3 tasks, TripPy then combines the losses into one as final loss to be used in backpropagation, as follows:

$$\text{loss} = \lambda \cdot \text{class_loss} + \frac{(1 - \lambda)}{2} \cdot (\text{token_loss} + \text{refer_loss}).$$

In some turns where no coreference classification is performed, only two losses are considered and the final loss is combined as follows:

$$\text{loss} = \lambda \cdot \text{class_loss} + (1 - \lambda) \cdot (\text{token_loss}).$$

3 Ablation Study

An ablation study is a method used to understand the contribution of individual components in a complex system, such as a machine learning model or algorithm. To perform an ablation study, we systematically remove or alter parts of the system and perform an evaluation. Comparing the evaluation results when certain parts are removed allows us to study how each part affects the overall performance and identify which components are essential and which may be redundant or even harmful. Ablation studies are crucial for interpreting model behavior, validating design choices, and guiding future improvements. They help ensure that performance gains are due to meaningful innovations rather than incidental design choices or spurious correlations.

4 Exercises

4.1 Codebase

Fork the following base project from git:

<https://git.hhu.de/sds/trippy>

Please set the visibility to **private**. To ensure that your repository is visible for grading, add user **lubis** as a project member.

4.2 Tasks

The script used in tasks 1 and 2 is

`trippy/modeling.dst.py`

Task 1: TripPy gates You will define the classification heads, i.e. gates, needed to perform state tracking.

1. For each classification task (class type, span, and refer), define a gate as part of the model. Implement your solution inside the box labeled `TODO DST_TASK1`. Note that each slot requires its own gates, i.e. the gates need to be defined for each slot.
 - You may use `self.add_module` function to add the gates to the model, more info here. An example of the refer slot is given in the code.
 - For each gate, identify the input and output size.
 - For class and refer predictions, in addition to the embeddings we use auxiliary features as part of the input.
 - For span prediction, we predict the indices of both start and end position at once.

Task 2: Loss computations You will complete the forward method of TripPy, first making a prediction using the model output and then calculating the loss based on that prediction. Implement your solution inside the box labeled `TODO DST_TASK2`, from 2a to 2f.

1. Implement the computation of logits for class type, token spans, and referral.
 - You may use `self.getattr(self, <gate_name>)` to access each of the gates.
 - Slot type and refer gates utilize the embedding of [CLS] token
 - Span gate uses the full sequence of token embeddings
2. Implement the computation of loss for class prediction and span prediction.
 - For each task, you will need to identify the loss function, the label, and prediction.
3. Combine the losses using the weighting strategy formulated in Section 2.2.
 - Use the weighting hyperparameter `self.class_loss_ratio` to mix the losses

Task 3: Train TripPy After implementing the functions above, set up an execution script to train a full TripPy model. The following starter script is provided

```
D0.simple_mini
```

Adjust the hyperparameters and run the model training by executing the script. For each training runs, the predictions, results, and logs are automatically saved into the corresponding experiment folder.

Task 4: Ablation Study Perform an ablation study by systematically removing parts of the input features. Train 3 separate models which take as input:

1. Full input features (I.e. Task 3)

2. Input features without auxiliary features
3. Input features without dialogue history and auxiliary features

Tip: Keep track of the experiments by using descriptive names or separate execution scripts to keep the results organized.

4.3 Results and Discussion

In your final report, write a short report covering, but not limited to, the following points. Additional insights, analysis, and discussion are highly encouraged.

- What is the input and output of TripPy? What are the auxiliary features used in TripPy? What is the intuition behind them?
- Plot and analyze the training and validation loss of the model training from task 3. You may average the losses per batch, iteration, or epoch. What are your findings?
- Present a table of the joint goal accuracy of the ablation study from task 4 on the validation and test sets. You may refer to Table 4 in [1] as reference. Discuss and analyze your result.

Push your completed scripts into your git repository and put the link to your repository in your report.

References

- [1] HECK, M., VAN NIEKERK, C., LUBIS, N., GEISHAUSER, C., LIN, H.-C., MORESI, M., AND GAŠIĆ, M. Trippy: A triple copy strategy for value independent neural dialog state tracking. In *SIGdial* (2020).